

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

* Teste Global de Época de Recurso (30 de Janeiro de 2026) - Duração 1h30

NÚMERO _____ NOME _____

Os Grupos 1 e 2 devem ser respondidos no enunciado. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

GRUPO 1 [4 valores]

Selecione a única opção verdadeira.

1. Numa função `async`, o que acontece se retornar um valor que não é uma `Promise` (por exemplo, um número ou uma `string`)?
A. O valor é automaticamente envolvido (*wrapped*) numa `Promise` resolvida.
B. A função deixa de ser assíncrona e passa a comportar-se como uma função comum.
C. O valor é descartado e a função retorna `'undefined'` assincronamente.
D. O JavaScript lança um `TypeError` porque a função deveria retornar uma `Promise` explicitamente.
2. Como a função `Math.max` deve ser chamada com a função `withLogging` para que seus argumentos e valor de retorno sejam apresentados na consola?
A. `withLogging(Math.max)(3, 5, 1);`
B. `withLogging(Math.max, 3, 5, 1);`
C. `withLogging(Math.max, 3, 5, 1)();`
D. `Math.max(withLogging(3, 5, 1));`
3. Ao analisar o URI `http://api.isel.pt:8080/v1/users?search=10`, qual das seguintes afirmações é verdadeira?
A. A query string neste URL é `users?search=10`
B. O caminho neste URL é `/v1/users`
C. A query string neste URL é `search`
D. O caminho neste URL é `/v1/users?search=10`
4. Num pedido HTTP em que contém dados no *body* em formato JSON, qual é o *header* obrigatório?
A. `Content-type: application/json`
B. Nenhum
C. `Set-Cookie: application/json`
D. `Cookie: application/json`
5. No contexto do Express, o argumento `next` em `app.use((req, res, next) => {...})` é:
A. Uma função que, ao ser chamada, passa o controlo para o próximo middleware.
B. Um objeto do tipo `Next` que contém o próximo middleware.
C. Uma função que deve ser implementada para tratar o ciclo Request-Response do Express.
D. Um objeto que possui um método `call` para chamar o próximo middleware.

```
function withLogging(fn) {  
  return function(...args) {  
    console.log('Arguments:', args);  
    const res = fn(...args);  
    console.log('Return:', res);  
    return res;  
  };  
}
```

6. Considere o seguinte trecho de um ficheiro YAML/JSON representando uma especificação OpenAPI. Qual dos seguintes objetos no formato JSON é **VÁLIDO** como *body*?

- A. `{"name": "Maria"}`
- B. `{"name": "Maria", "age": "9"}`
- C. `{ "age": 9 }`
- D. `[{"name": "Maria"}, {"age": 9}]`

```
requestBody:  
  content:  
    application/json:  
      schema:  
        type: object  
        required:  
          - name  
        properties:  
          name:  
            type: string  
          age:  
            type: integer
```

```
{ "requestBody": {  
  "content": {  
    "application/json": {  
      "schema": {  
        "type": "object",  
        "required": ["name"],  
        "properties": {  
          "name": {"type": "string"},  
          "age": {"type": "integer"}  
        }  
      }  
    }  
  }  
}}
```

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

* Teste Global de Época de Recurso (30 de Janeiro de 2026) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 2 [4 valores]

Faça a correspondência entre a opção que está no lado esquerdo da tabela e a frase que a complementa no lado direito, escrevendo o número correspondente a essa opção. Por exemplo, se considerar que a opção 1 do lado esquerdo corresponde à 3ª frase do lado direito, coloque 1 na 3ª linha do lado direito.

1. No contexto do HTTP:

1	O status code 301	4	corresponde a um erro no servidor.
2	O status code 200	2	indica que o pedido foi processado com sucesso.
3	O status code 400	1	corresponde a um redirecionamento.
4	O status code 500	3	corresponde a um erro no pedido.

2. Em `<div class="container"> John, <p> say <strong id="highlight"> hello </strong> </p></div>`, o selector em CSS:

1	<code>#highlight {color: blue}</code>	3	não altera a cor do texto.
2	<code>p {color: blue}</code>	2	apenas "say hello " fica com o texto em azul.
3	<code>.container {color: blue}</code>	1	apenas " hello " fica com o texto em azul.
4	<code>#container {color: blue}</code>	4	"John, say hello " fica com o texto em azul.

3. No HTTP:

1	O método PUT	4	é safe e idempotente.
2	O método POST	3	não é safe e é idempotente.
3	O método DELETE	2	não é safe e nem é idempotente.
4	O método GET	1	nem sempre pode ser usado para criar um recurso.

4. Numa aplicação Express, considere as seguintes rotas associadas a funções middleware:

```
app.get("/:y/x", funcA);  
app.post("/:y/x", funcB);  
app.use("/x", funcC);
```

1	O pedido POST /z/x	4	chama a função funcC.
2	O pedido POST /z/z	3	chama a função funcA.
3	O pedido GET /z/x	2	não chama nenhuma das funções apresentadas.
4	O pedido GET /x/z	1	chama a função funcB.

5. Numa aplicação Express, considere o seguinte código:

```
import express from "express";  
const app = express();  
app.use(express.json());
```

1	use	3	retorna uma função que não é um middleware.
2	express.json()	4	é um objeto que representa a aplicação.
3	O módulo express	1	tem como parâmetro uma função.
4	app	2	retorna um middleware.

6. Em HTML:

1	<code><input type="submit"></code>	4	define o nome a enviar no pedido associado ao valor do input.
2	O elemento form	1	representa um botão para o envio dos dados do formulário.
3	<code><form method="POST"></code>	3	indica que os dados do formulário são enviados no corpo do pedido.
4	O atributo name num input	2	contém todos os elementos do formulário.

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

Teste Global de Época de Recurso (30 de Janeiro de 2026) - Duração 1h30

GRUPO 3 [3.5 valores]

1. [3.5] Considere o seguinte código em JavaScript que pode apresentar 4 tipos de mensagens na consola: W, V, S e E.

```
1 async function myFetch(url, timeout) {
2   let isFinished = false;
3   setTimeout(() => { if (!isFinished) {console.log(`W: ${timeout}ms...`); }}, timeout);
4   try {
5     const resp = await fetch(url);
6     isFinished = true;
7     if (!resp.ok) return Promise.reject(`S: ${resp.status}`);
8     const respJson = await resp.json();
9     console.log(`V: ${respJson.value}`);
10    return respJson;
11  } catch (error) {
12    isFinished = true; return Promise.reject(`E: ${error}`);
13  }
14 }
15 myFetch('https://api.chucknorris.io/jokes/random', 200).catch(console.log);
```

- a. [1.5] Apresente o output para os seguintes casos:
- O servidor da API retorna status code 200 em 300 ms.
 - O servidor da API retorna status code 404 em 100 ms.
- b. [2] Crie uma versão alternativa utilizando apenas .then() e .catch() em vez de async/await e try/catch, que apresente o mesmo resultado.

GRUPO 4 [3 valores]

Complete os espaços em branco de um programa em JavaScript, que cria um mecanismo de monitorização que permite acompanhar a utilização e desempenho de funções. Para isso, implemente a função monitor(fn) que recebe uma função (síncrona ou assíncrona) e devolve um objeto com duas propriedades: wrapped (a versão instrumentada da função) e metrics (função que retorna as métricas). As métricas incluem o número total de chamadas (calls), o número de falhas (failures) e o tempo médio de execução apenas das chamadas bem-sucedidas (avgTime).

```
1 async function testFn(x) {
2   if (x % 2 === 0) return x * 2;
3   else throw new Error("fail");
4 }
5 const monitored = monitor(testFn); // Versão monitorizada
6 // Exemplo de utilização
7 try { await monitored.wrapped(2); } catch {} // Sucesso
8 try { await monitored.wrapped(3); } catch {} // Falha
9 try { await monitored.wrapped(4); } catch {} // Sucesso
10 try { await monitored.wrapped(5); } catch {} // Falha
11 console.log(monitored.metrics()); // Obter métricas
12 // Output: { calls: 4, failures: 2, avgTime: 101.5 }
```

```
1 function monitor(fn) {
2   let calls = 0, failures = 0, successTime = 0, successes = 0;
3   async function wrappedFn(...args) {
4     _____ // 1;
5     const start = Date.now();
6     try {
7       const result = await _____; // 2
8       successTime += Date.now() - start; successes++;
9       return result;
10    } catch (e) { _____; // 3 throw e; }
11  }
12  function getMetrics() {
13    return { calls, failures, avgTime: successes ? successTime / successes : 0 };
14  }
15  return { wrapped: _____, // 4 metrics: _____ // 5 };
16 }
```

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

Teste Global de Época de Recurso (30 de Janeiro de 2026) - Duração 1h30

GRUPO 5 [5.5 valores]

1. [5] Complete os espaços em branco de uma aplicação em Node.js com Express que armazena em memória as últimas **n** ações de cada utilizador (identificado por token simples no header **X-User-Token**). Quando o limite **n** é ultrapassado para um utilizador, a ação mais antiga deste utilizador é removida. A aplicação mantém um objeto **userHistories** onde cada chave é um token de utilizador e o valor é um array com as suas ações. Um middleware global regista cada pedido (método HTTP, rota e timestamp) associado ao utilizador identificado pelo header. Se o header não estiver presente, o pedido não é registado. A aplicação deve expor três rotas:
 1. **GET /history**
Retorna todas as ações do utilizador atual (identificado pelo header **X-User-Token**) em formato JSON, ordenadas da mais recente para a mais antiga.
 2. **GET /history/:idx**
Recebe o parâmetro **idx** na path que indica o índice da ação a retornar. Quando **idx** é 0, retorna a ação mais recente; quando é 1, a penúltima; e assim sucessivamente.
 3. **DELETE /history**
Limpa todo o histórico do utilizador atual.
- a) [1] Complete a implementação do módulo **services.mjs**.
- b) [1.5] Complete a implementação do módulo **middleware.mjs**.
- c) [3] Complete a implementação do módulo **server.mjs**.

server.mjs

```
1 import express from "express";
2 import createHistoryMiddleware from "../middleware.mjs";
3 import { getUserHistory, getActionAtIndex, clearUserHistory } from "../services.mjs";
4
5 const app = express();
6 const port = 3000;
7
8 app.use(express.json());
9
10 app.use((req, res, next) => {
11   const userToken = _____ // 1
12   if (userToken) {
13     req.token = userToken;
14     _____ // 2
15   }
16   res.status(____).json({ error: "Token missing" }); // 3
17 })
18
19 // Keep the last 5 user actions
20 app.use(_____); // 4
21
22 app._____("/history", (req, res) => { // 5
23   const history = _____; // 6
24   res.json(history);
25 });
26
27 app.get("/history/:idx", (req, res) => {
28   const idx = parseInt(_____); // 7
29   const action = getActionAtIndex(req.token, idx);
30   if (!action) { return res.status(404).json({ error: "Action not found" }); }
31   _____; // 8
32 });
33
34 app._____("/history", (req, res) => { // 9
35   clearUserHistory(req.token);
36   res.json({ message: "History cleared" });
37 });
```

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

Teste Global de Época de Recurso (30 de Janeiro de 2026) - Duração 1h30

```
38
39 // .. Other application routes
40 app.listen(port, () => {
41   console.log(`Server running http://localhost:${port}`);
42 });
```

services.mjs

```
1  const userHistories = {};
2
3  export function addAction(userToken, action, n) {
4    if (!userHistories[userToken]) { userHistories[userToken] = []; }
5    _____; // 1
6    if (userHistories[userToken].length > n) { userHistories[userToken].shift(); }
7  }
8
9  export function getUserHistory(userToken) {
10   return userHistories[userToken] || [];
11 }
12
13 export function getActionAtIndex(userToken, idx) {
14   const history = getUserHistory(userToken);
15   return _____; // 2
16 }
17
18 export function clearUserHistory(userToken) {
19   delete _____; // 3
20 }
```

middleware.mjs

```
1  import { addAction } from "../services.mjs";
2
3  function createHistoryMiddleware(n) {
4    return (_____ ) => { // 1
5      if (req.token) {
6        const action = {
7          method: req._____, // 2
8          route: req.path,
9          timestamp: new Date().toISOString()
10        };
11
12        addAction(_____); // 3
13      }
14
15      next();
16    };
17  }
18
19  export default createHistoryMiddleware;
```

Os docentes,
Fernanda Passos, Filipe Freitas, José Faustino e Luís Falcão